

Commandes DevOps - Session du 2026-08-26

1. Environnement WSL

- `wsl --list --verbose` : lister les distributions WSL et leur version.
- `wsl -d Debian` : se connecter à la distribution Debian.
- `wsl -d Debian -u root` : se connecter en tant que root sur Debian.
- `exit` : quitter une session Linux.

2. Gestion des utilisateurs (SysOps)

- `passwd admin` : changer le mot de passe de l'utilisateur admin (nécessite d'être root).
- `sudo usermod -aG docker $USER` : ajouter l'utilisateur courant au groupe docker.

3. Gestion des paquets (Debian)

- `sudo apt update` : rafraîchir la liste des paquets disponibles.
- `sudo apt install -y ca-certificates curl` : installer les dépendances pour ajouter un dépôt.
- `sudo apt install -y docker-ce docker-ce-cli containerd.io` : installer Docker (déjà présent).

4. Docker

- `docker --version` : vérifier la version de Docker.
- `sudo service docker status` : vérifier que le service Docker tourne.
- `docker run hello-world` : tester Docker en exécutant un conteneur de démonstration.

5. Ajout d'un dépôt sécurisé

- `sudo curl -fsSL https://download.docker.com/linux/debian/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg` : télécharger et installer la clé GPG de Docker.
- `echo "deb [arch=amd64 signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] https://download.docker.com/linux/debian trixie stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null` : ajouter le dépôt Docker (version corrigée avec trixie).

Notes personnelles

- Ne pas utiliser `wsl` dans un terminal Linux (commande réservée à PowerShell).

- Pour réinitialiser un mot de passe perdu : se connecter en root avec `wsl -d Debian -u root`, puis `passwd admin`. ## 6. Docker et Docker Compose (Phase 4)
- `docker build -t portfolio-auth-api .` : construire l'image Docker.
- `docker run -d --name portfolio-api -p 8000:8000 portfolio-auth-api` : lancer un conteneur (test).
- `docker logs portfolio-api` : consulter les logs du conteneur.
- `docker exec -it portfolio-api /bin/sh` : entrer dans un conteneur en mode interactif.
- `docker compose up -d` : lancer la stack avec PostgreSQL et l'API.
- `docker compose down` : arrêter et supprimer les conteneurs.
- `docker compose build --no-cache` : reconstruire les images sans utiliser le cache.
- `docker ps` : lister les conteneurs en cours d'exécution.
- `docker ps -a` : lister tous les conteneurs (y compris arrêtés).
- `docker rm <container>` : supprimer un conteneur.
- `docker rmi <image>` : supprimer une image.
- `docker network ls` : lister les réseaux Docker.
- `docker network inspect <network>` : inspecter un réseau Docker. ## 6. Gestion des conteneurs et réseau (Session Septembre 2026)

Inspection et diagnostic réseau

- `docker network ls` : Lister les réseaux Docker.
- `docker network inspect [nom_du_reseau]` : Voir les détails d'un réseau (IP des conteneurs, sous-réseau).
- `docker inspect portfolio-db | grep IPAddress` : Récupérer l'IP d'un conteneur spécifique.
- `docker exec -it portfolio-adminer ping db` : Tester la connectivité réseau entre deux conteneurs (ex: Adminer → DB).
- `hostname -I` : Trouver l'IP de la distribution WSL depuis l'intérieur de celle-ci.

Gestion de la base de données (PostgreSQL)

- `docker exec -it portfolio-db psql -U postgres -d portfolio_db -c "\l"` : Lister les bases de données.
- `docker exec -it portfolio-db psql -U postgres -c "DROP DATABASE portfolio_db;"` : Supprimer une base (nécessite de couper les connexions).
- **Gestion des connexions actives :**
 - `docker exec -it portfolio-db psql -U postgres -c "UPDATE pg_database SET datallowconn = 'false' WHERE datname = 'portfolio_db';"` : Mettre la DB en mode maintenance.
 - `docker exec -it portfolio-db psql -U postgres -c "SELECT pg_terminate_backend(pid) FROM pg_stat_activity WHERE`

```
datname = 'portfolio_db' AND pid <> pg_backend_pid();" :
```

Forcer la déconnexion de tous les utilisateurs.

Sauvegarde et Restauration (Disaster Recovery)

- `docker exec -it portfolio-db pg_dump -U postgres -d portfolio_db > backup.sql` : Sauvegarder la base dans un fichier.
- `docker exec -i portfolio-db psql -U postgres -d portfolio_db < backup.sql` : Restaurer une base à partir d'un fichier de sauvegarde.

Multi-environnements (Dev vs Prod)

- `docker compose -f docker-compose.prod.yml up -d` : Lancer la stack avec un fichier Compose alternatif (ex: production).
- `docker compose -f docker-compose.prod.yml down` : Arrêter la stack avec le fichier alternatif.

Authentification JWT (tests manuels)

- `curl -X POST "http://localhost:8000/auth/login" -H "Content-Type: application/json" -d '{"email":"test@mail.com","password":"pass"}'` : Obtenir un token.
- `curl -X GET "http://localhost:8000/auth/me" -H "Authorization: Bearer [TOKEN]"` : Tester une route protégée avec le token récupéré.

7. Notes personnelles (Nouvelles réflexions)

- **Erreur Float not defined** : Penser à importer Float dans sqlalchemy lors de l'ajout de colonnes décimales.
- **Erreur JSON decode error (422)** : Ne JAMAIS envoyer deux objets JSON à la suite dans le body d'une requête Swagger.
- **Erreur \$ dans .env** : Les variables d'environnement contenant des \$ ou ! doivent être échappées ou mises entre guillemets simples dans le fichier .env.
- **Différence Dev/Prod** : En Dev, on utilise `--reload` et des volumes (bind mounts). En Prod, on build l'image et on évite ces montages pour la sécurité et les performances.